SAGEA-fluid User Manual (Version1.0)

Weihang Zhang[1], Shuhao Liu[1], Fan Yang[2,*], Shin-Chan Han[3], and Ehsan Forootan[2,4]

[1]School of Physics, Huazhong University of Science an d Technology, Wuhan 430074, China

[2]Geodesy Group, Department of Sustainability and Planning, Aalborg University, Aalborg 9000, Denmark

[3]Division of Geodetic Science, School of Earth Sciences, The Ohio State University, Columbus 43210, USA

[4]School of Geographical Sciences, University of Bristol, University Rd, Bristol BS8 1SS, United Kingdom

Corresponding author: Fan Yang, fany@plan.aau.dk

1 Overview

SAGEA-fuid is an open-source toolbox to estimate geophysical effects like self-attraction and loading (SAL), geocenter motion (GCM), Earth orientation parameters (EOPs) which includes both polar motion and length-of-day variation. It is built on an advanced object-oriented Python framework, aiming to provide user-friendly services to especially the non-expert users in geodetic community.

Specifically, SAGEA focuses on processing GRACE Level-2 data (monthly gravity fields) into Level-3 products (physical variable for a specific study) and performing related error analysis. SAGEA-fluid further uses the Level-3 products derived from SAGEA to estimate key geophysical effects including self-attraction and loading, geocenter motion and Earth orientation parameters, which correspond to GRACE Level-4 products. SAGEA-fluid is linked with SAGEA because SAGEA can provide the necessary GRACE input data for its computation, but SAGEA-fluid also supports a much wider variety of data sources beyond GRACE products. The hierarchical relationship between SAGEA and SAGEA-fluid, along with their respective roles in GRACE data processing, is illustrated in Fig. UM1.

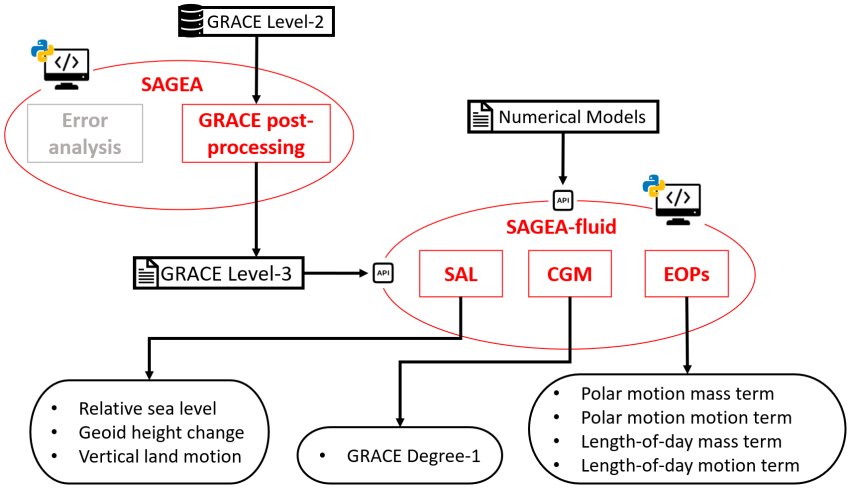

Fig. UM1. Schematic of the relationship between SAGEA and SAGEA-fluid

To clarify, the three major modules, i.e., SAL, GCM and EOP, are not entirely independent. While their code development is independent, but these three modules may have to communicate and cooperate for accomplishing a specific task. The major interactions between these modules include:

- GCM relies on SAL for establishing the GRACE-OBP approach.
- EOP relies on SAL for computing the sea-level angular momentum (SLAM) component.

In addition, as illustrated in the framework (see Fig. 1 in the original manuscript), these modules almost shared all the input loading:

- By the source of loading: the loading can be from atmosphere, hydrology, ocean, icesheet and even Earthquake (this has not been addressed but will be considered in future updates).
- By the state of the loading: flux, and storage/pressure. But be aware that only EOP requires the flux data.

In summary, SAGEA-fluid preserves clear modularity while maintaining strict numerical and physical consistency across all components through a shared foundational architecture and a unified data structure.

The major limitation and challenge, from both software engineering and science application, include:

- The supported data format is not rich enough. For example, we currently do no support for .grib climate model files, while which is quite common these days.
- Lack of a complete spectral method for SAL, which may restrict high-resolution coastal sea level studies, but this is yield to more investigation.
- The GCM module is currently only applicable to monthly GRACE degree-1 terms; whether it can also work well for daily or shorter term GRACE product (like weekly) remains uncertain but will be investigated soon, particularly for the next generation gravity mission that aims to release 5-daily gravity product.
- EOP computation from Earth's surface dynamics are still deviating much from the observed EOP (like those from GNSS). While this is a common issue, how to close and interpret these discrepancies are likely of great interest in science community.

2 Dependence

SAGEA-fluid is fully decoupled from the parent SAGEA platform at the codebase level. This strict decoupling ensures that any updates, modifications, or iterations to the SAGEA

platform do not impact the functionality, installation, or operation of SAGEA-fluid, and vice versa (i.e., updates to SAGEA-fluid do not affect the parent SAGEA platform). To maintain this decoupled architecture, we have established a formal agreement with the core developers of the SAGEA platform to freeze the predefined interface specifications between the two systems; this eliminates potential compatibility issues arising from shared codebases or library dependencies, and guarantees independent deployment of SAGEA-fluid. In other words, SAGEA-fluid communicates with SAGEA indirectly through a proxy interface, which grantees the independency.

Except for SAGEA, SAGEA-fluid also relies on a few common Python libraries for enhancing numerical computations. The full list of these core geodetic and numerical libraries is explicitly presented in Table 1.

Table 1. The essential packages of SAGEA-fluid environment.

Python package	Version
geopandas	0.14.4
h5py	3.12.1
netCDF4	1.7.2
numpy	1.26.0
scipy	1.10.0-rc1
shapely	2.0.0
tqdm	4.66.3
wget	3.2
requests	2.23.3
pandas	2.2.3
xarray	2024.7.0

Regarding potential installation conflicts (with SAGEA), we acknowledge that SAGEA-fluid and SAGEA have a few common dependencies. To mitigate the risk of conflict and ensure smooth installation, we provide explicit version recommendations:

- `numpy`: version 1.26.0 or higher;
- `xarray`: version 2024.7.0 or higher;

- Python interpreter: version 3.9 or higher.

For visualization, SAGEA-fluid does not enforce specific plotting libraries, allowing users to choose based on their needs. Commonly used options include `matplotlib`, `cartopy`, and `pygmt`. The `requirements.txt` file included in the source code already contains `matplotlib` and `cartopy`, which can be directly installed via the `requirements.txt` file.

3 Installation

The SAGEA-fluid project is publicly available at `https://github.com/NCSGgroup/SAGEA-fluid`. It is developed using Python 3.9, and all necessary dependencies are specified in the accompanying `requirements.txt` file. To ensure a consistent computing environment, we recommend installing SAGEA-fluid using conda. The setup procedure is as follows:

1. `conda create -n SAGEA-Fluid python=3.9`
2. `conda activate SAGEA-Fluid`
3. `pip install -r requirements.txt`

Users may alternatively install the required packages directly in their existing environment by using the command `python -m pip install -r requirements.txt`. Since SAGEA-fluid is built upon the SAGEA framework, users may refer to the installation and configuration instructions of SAGEA (Liu et al., 2025) for additional guidance.

4 Sample data

To successfully estimate the self-attraction and loading (SAL) effect, geocenter motion (GCM), and Earth orientation parameters (EOPs) using SAGEA-fluid, several essential sample datasets are required. These datasets should be placed in the directory `/SAGEA-fluid/data/` before running the demonstrations. The sample data are available at `https://doi.org/10.6084/m9.figshare.30104062.v6`. After downloading, place them directly in the root directory so that users can proceed with further scientific analysis.

5 Quick start

For the convenience of users, several demonstration scripts are provided to help estimate SAL, GCM, and EOPs using SAGEA-fluid. These demos can be found in the directory

`/SAGEA-fluid/demo/`. To ensure broad applicability, three groups of demos are designed to address the main analysis tasks of users:

1. **demoSAL** is used for estimating the SAL effect. It contains two examples. The first example, `demo_NM()`, demonstrates how to compute SAL based on numerical models using SAGEA-fluid. The second example, `demo_GO()`, shows how to estimate SAL with GRACE GSM data as input. By running demoSAL, users can obtain SAL results separately from the atmosphere, ocean, hydrology, and ice sheets;
2. **demoGCM** provides two examples to illustrate the estimation of GRACE low-degree terms. The first example, `demo_GCM()`, generates geocenter motion products in both GSM-like form (containing contributions from hydrology and ice sheets only) and full-geocenter form (including contributions from the atmosphere and ocean). For detailed explanations of these two forms, refer to Swenson et al. (2008). The second example, `demo_J2()`, enables users to estimate GRACE C_{20} Stokes coefficients for the period from January 2009 to December 2009;
3. **demoEOP** contains four examples that illustrate how to estimate both the mass term and the motion term of polar motion as well as length-of-day variation. These examples are called `demo_PM_mass_term()`, `demo_PM_motion_term()`, `demo_LOD_mass_term()`, and `demo_LOD_motion_term()` in the Python script.

These demos allow users to efficiently reproduce key results and further extend the applications of SAGEA-fluid in related geophysical studies.

6 Tutorial

The tutorials below assume that the necessary Python libraries have already been imported. The import statements for the three core modules are as follows:

- `from pysrc.SAL.SeaLevelEquation import PseudoSpectralSLE/SpatialSLE`
- `from pysrc.GCM.GeocenterMotion import GeocenterMotion`
- `from pysrc.EOP.EarthOrientaition import EOP`

To start with, we provide important clarifications regarding potential input data issues that may cause incorrect results or runtime failures. When loading GRACE GSM products, differences exist in the file-naming conventions between official releases (CSR, GFZ, JPL) and non-official GRACE solutions. Directly using the standard method of SAGEA load

for non-official products named by year and month may lead to loading errors. For this reason, SAGEA-fluid provides dedicated functions for loading non-official GSM datasets, which can be accessed via `pysrc.SaKits.loadFile.LoadICGEM`, including `load_HUSTGRACE`, `load_ITSG`, `load_SWPU`, and other product-specific methods.

In addition, atmospheric reanalysis, ocean model outputs, and other numerical models often contain a large number of variables. Users must have sufficient understanding of the input data to select exactly the required variables for computing the corresponding geophysical effects. In particular, the estimation of motion terms for Earth orientation parameters requires vertically structured data. SAGEA-fluid requires vertical layers to be ordered from the near-surface upward to the upper domain. For oceanic and hydrological profiles, the upper domain represents deeper layers away from the surface. Datasets provided in the reverse order must be reordered before input.

Beyond data loading, users may also encounter mixing different excitation sources for each module. To clarify the required inputs for each core module and avoid mixing, we provide a schematic overview of the data dependencies for SAL, GCM, and EOP calculations, as illustrated in Fig. UM2.

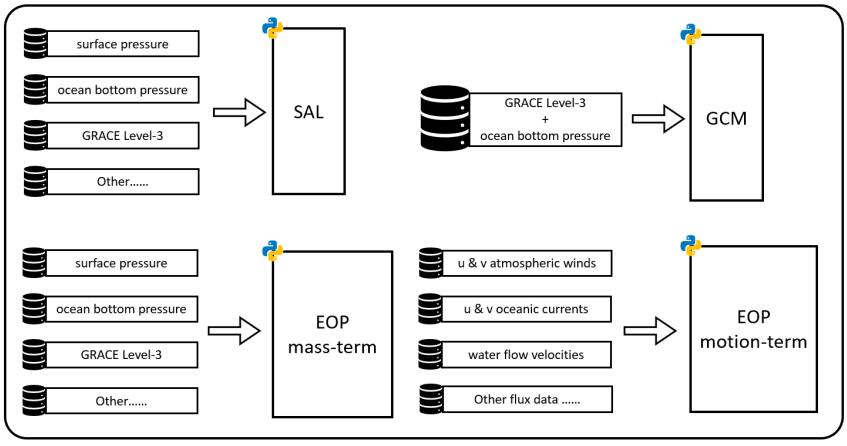

Fig. UM2. Schematic diagram of specific datasets for exciting the geophysical phenomena in SAGEA-fluid

As shown in the schematic, the SAL module and the EOP mass-term submodule both require surface pressure, ocean bottom pressure, GRACE Level-3 products, and other auxiliary datasets as primary inputs. The GCM module takes GRACE Level-3 products combined with ocean bottom pressure as its core input. For the EOP motion-term, the required

inputs are u and v components of atmospheric winds, oceanic currents, water flow velocities, and other flux data. This clear mapping of inputs to modules helps users correctly configure their workflows and ensures the physical consistency of the derived geophysical products.

Moreover, there are some practical guidance on dataset for user selection:

1. For self-attraction and loading effects calculation, due to temporal gravity field can represent the global terrestrial water storage change, thus GRACE GSM products from CSR, GFZ, and JPL are good choices for sea level fingerprint estimation.
2. For geocenter motion estimation, except for the GRACE GSM, the ocean model output we recommend using the monthly product GAD, which is used to estimate TN-13 (Sun et al., 2016).
3. For Earth orientation parameters (EOPs) computation, we suggest adopting the latest version of numerical models. For example, when using ECMWF products, ERA5 should be employed instead of the older ERA-Interim.

6.1 SAL

When estimating SAL using SAGEA-fluid, the procedure generally consists of three steps. The pseudo-spectral method is used here as an example.

- **Step 1: Import the input data.**

  `SAL = PseudoSpectralSLE(SH, lmax)`, where `SH` denotes the input spherical harmonic coefficients expressed in equivalent water height (EWH).
- **Step 2: Configure the model settings.**

  `SAL.setLatLon(lat, lon)` specifies the latitude and longitude associated with the input data;

  `SAL.setLoveNumber(lmax, method = LoveNumber_type)` selects the Love numbers to be applied. If this option is not specified, the default Love numbers from Wang et al. (2012) consistent with the input degree are used.
- **Step 3: Run the sea level equation.**

  `Results = SAL.SLE(mask, rotation=True, isLand=True)`, where `mask` specifies the ocean function, for which SAGEA-fluid provides a default. The `rotation` parameter controls whether rotational feedback is included. The `isLand` parameter indicates whether water exchange between land and ocean is allowed.

After solving, users may obtain different output variables such as `Input`, `RSL_SH`, `Quasi_RSL_SH`, `RSL`, `GHC`, `VLM`, or `mask`. For example, the spatial distribution of relative sea-level change (EWH, in meters) can be extracted using `Results['RSL']`.

SAGEA-fluid also supports a spatial-domain method. In this case, step 1 is replaced by `SAL = SpatialSLE(grid, lat, lon)`, which does not require separately setting latitude and longitude.

6.2 GCM

The estimation of GRACE geocenter motion follows a similar three-step workflow.

- **Step 1: Import the input data.**
  `GCM = GeocenterMotion(GRACE, OceanSH, GAD, lmax)`, where `GRACE`, `OceanSH`, and `GAD` are all the mass coefficients of surface loads.
- **Step 2: Configure the spatial resolution.**
  `GCM.setResolution(resolution)`, which ensures consistency between the input data and the ocean function. The default resolution is 1°.
- **Step 3: Run the GRACE-OBP.**
  `Results = GCM.Low_Degree_Term(mask, buffer, SAL=True, rotation=True)`, where `SAL` controls whether SAL effects are included, `rotation` determines whether rotational feedback is considered, and `buffer` specifies the spatial buffer width used for leakage correction (default 0 km).

Users may then retrieve the estimated low-degree terms as Stokes coefficients using `Results['Stokes']` or as mass coefficients using `Results['Mass']`. For convenience, geocenter motion can also be obtained directly via `Results = GCM.GSM_Like(mask, SAL, rotation, buffer)`, and individual geocenter components can be accessed through the X, Y or Z options (in meters).

6.3 EOPs

The estimation of Earth orientation parameters in SAGEA-fluid is performed separately for mass terms and motion terms. The relevant commands are listed below.

- **Mass term (spectral method).**

  `EOPs = EOP().PM_mass_term_SH(SH, isMas=False);`

  `EOPs = EOP().LOD_mass_term_SH(SH, isMs=False)`, where `SH` represents surface fluid spherical harmonic coefficients (Stokes coefficients). If `isMas` is True, polar motion is output in milliarcseconds; if `isMs` is True, length-of-day variation is in milliseconds.

- **Mass term (spatial method).**

  `EOPs = EOP().PM_mass_term(Ps, lat, lon, isMas=False);`

  `EOPs = EOP().LOD_mass_term(Ps, lat, lon, isMs=False)`, where `Ps` is the surface pressure field (Pa), and [`lat, lon`] correspond to the latitude and longitude of the input.

- **Motion term.**

  `EOPs = EOP().PM_motion_term(Us, Vs, lat, lon, levPres, type, isMas=False);`

  `EOPs = EOP().LOD_motion_term(Us, lat, lon, levPres, type, isMs=False)`, where [`Us`, `Vs`] are the zonal and meridional wind or current components (m s^{-1}), and `levPres` denotes atmospheric pressure levels or ocean layer depths. The `type` parameter specifies whether atmospheric or oceanic angular momentum is used.

Users may extract specific EOP components such as χ_1, χ_2, or χ_3 using statements like `EOPs['chi1']`, `EOPs['chi2']` or `EOPs['chi3']`. Specifically, `EOPs['chi1']` and `EOPs['chi2']` are used to retrieve the components for polar motion, while `EOPs['chi3']` is used to retrieve the component for length-of-day (LOD).

It should be noted that the `type` parameter must be assigned using the enumeration class in SAGEA-fluid. To achieve this, users first need to import the enumeration class with the following syntax: `from pysrc.SaKits.Setting.EnumClasses import EnumClass`. After importing, the desired excitation source can be set (e.g., `type=EnumClass.EAMtype.AAM`).

References

Liu, S., Yang, F., & Forootan, E. (2025). SAGEA: A toolbox for comprehensive error assessment of GRACE and GRACE-FO based mass changes. *Computers & Geosciences*, *196*, 105825. doi: 10.1016/j.cageo.2024.105825

Sun, Y., Riva, R., & Ditmar, P. (2016). Optimizing estimates of annual variations and trends in geocenter motion and J2 from a combination of GRACE data and geophysical models. *Journal of Geophysical Research: Solid Earth*, *121*(11), 8352-8370. doi: 10.1002/2016JB013073

Swenson, S., Chambers, D., & Wahr, J. (2008). Estimating geocenter variations from a combination of GRACE and ocean model output. *Journal of Geophysical Research: Solid Earth*, *113*, B08410. doi: 10.1029/2007JB005338

Wang, H., Xiang, L., Jia, L., Jiang, L., Wang, Z., Hu, B., & Gao, P. (2012). Load Love numbers and Green's functions for elastic Earth models PREM, iasp91, ak135, and modified models with refined crustal structure from Crust 2.0. *Computers & Geosciences*, *49*, 190-199. doi: 10.1016/j.cageo.2012.06.022